README

Setup & run instructions

Multi-Agent Incident Response System — Track 2 Capstone
github.com/bgolsen/agents-final-project

Multi-Agent Incident Response System

Capstone project, Track 2 (Multi-Agent Incident Response System). See `ARCHITECTURE.md` for the full design writeup, diagrams, and technical analysis, and `SCRIPT.md` for the demo video script. This README is setup/run instructions only. PDF versions of all three (`README.pdf`, `ARCHITECTURE.pdf`, `SCRIPT.pdf`) are included at the repo root for easier offline reading; regenerate them after editing the `.md` sources with `python .pdfbuild/build_pdfs.py` (requires Chrome and `npx` for mermaid-cli — see the script for details).

A simulated production incident comes in as an alert. Four **Google ADK** agents, each its own **A2A** server, collaborate under a rule-based Coordinator to triage it, diagnose root cause against a knowledge base of past incidents, plan a risk-ranked remediation, stop for **human approval**, execute the approved plan, and write a post-mortem. **n8n** fronts the system for alert intake/routing; **LangSmith** traces the full reasoning chain.

Demo video: `<add your unlisted YouTube/Loom link here before submitting>`

Prerequisites

- Python 3.11+ (developed on 3.12)
- (Optional but recommended) a Google AI Studio API key for real Gemini-driven reasoning. **Without one, the system still runs end-to-end** — every agent has a deterministic rule-based fallback (see `ARCHITECTURE.md` §6) that fires automatically when the LLM call fails, which is exactly how this project was developed and tested.
- (Optional) a LangSmith API key to see traces. Without one, tracing is a documented no-op (see `ARCHITECTURE.md` §3).
- (Optional) Docker, to run n8n for the alerting/routing workflow via `docker compose up -d` (see below) rather than the CLI demo posting directly to the coordinator. Free/open-source, no n8n.io account needed.

Setup

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
# macOS/Linux:
source .venv/bin/activate

pip install -r requirements.txt      # or requirements-dev.txt to also run tests
cp .env.example .env                 # then fill in GOOGLE_API_KEY / LANGSMITH_API_KEY
```

Run the demo (single command)

```
python -m incident_response.run_incident
```

This starts the 4 specialist A2A servers (ports 8001-8004) and the coordinator API (port 8110) as subprocesses, lets you pick one of 3 canned incident scenarios, runs triage → diagnosis → planning, prints each agent's structured output, **stops and prompts you** to approve/reject/modify the remediation plan (the HITL checkpoint), then executes and prints the post-mortem. Processes are torn down automatically on exit. Each subprocess's own stdout/stderr (model calls, ADK's internal retry-on-validation-failure noise, etc.) is written to `logs/<agent>.log` instead of the terminal, so the demo output stays readable — check those files if something looks wrong.

Useful flags:

```
python -m incident_response.run_incident --scenario auth-cert-expiry   # skip the menu
python -m incident_response.run_incident --auto-approve                # non-interactive (scripted demo/CI)
python -m incident_response.run_incident --no-spawn                    # reuse already-running servers
```

The three canned scenarios (`incident_response/data/scenarios.py`) each inject a different simulated failure so you can see error handling in action:

scenario_id	what's wrong	what fails once (chaos injection)
`checkout-latency-spike`	downstream payment-gateway timeouts cascading	the log API
`auth-cert-expiry`	expired TLS certificate	the knowledge-base search
`payment-db-pool-exhaustion`	leaked DB connections after a deploy	the metrics API, and the first remediation-execution attempt

Live incident dashboard & report

Terminal output only shows so much. While the system is running (via either `run_incident.py` or the n8n workflow below), open **http://localhost:8110/dashboard** in a browser — it lists **every incident the coordinator has ever handled**, not just the current one: on startup the coordinator reloads every `runs/<incident_id>.json` it has saved, so history survives restarts, with each one's status (`awaiting_approval`, `closed`, `rejected`, ...) color-coded and the list auto-refreshing while any incident is still in flight. Click into one for the full report at `/incidents/{id}/report`: each agent's actual structured output (triage anomalies, the diagnostic agent's sub-question decomposition and ranked hypotheses with cited evidence and confidence, the remediation goal/step plan with risk and rollback, the execution log, the postmortem) rendered as readable HTML, with a raw-JSON toggle at the bottom for the full record. This page updates in real time as each agent finishes — leave it open during a live demo and watch each section fill in — because the coordinator mutates the same in-memory `IncidentState` object that the page reads, so progress is visible mid-run, not just after the incident closes.

The HITL checkpoint itself lives on this page, not just in a terminal. When an incident reaches `awaiting_approval`, the report page shows an actual form — your name, notes, a dropdown of which goal to run if you modify — with Approve / Modify / Reject buttons that post straight to the coordinator and redirect back to the (now-updated) report. This is the only way to act on an incident that wasn't started from `run_incident.py`'s interactive prompt — e.g. anything filed through n8n or curl — since the dashboard has no terminal to prompt in. (The page intentionally stops auto-refreshing while a decision is pending, so a timed reload can't wipe out what you're mid-typing.)

Run the pieces individually

```
# one terminal per agent, or use scripts/start_agents.ps1 on Windows
python -m incident_response.a2a_server monitoring     # :8001
python -m incident_response.a2a_server diagnostic      # :8002
python -m incident_response.a2a_server remediation     # :8003
python -m incident_response.a2a_server postmortem      # :8004
python -m incident_response.api                        # :8110 (coordinator + HITL API)

# then drive it with curl, or:
python -m incident_response.run_incident --no-spawn
```

Each agent's A2A card is inspectable directly: `curl http://localhost:8001/.well-known/agent-card.json`

Coordinator API surface (also used by n8n):

Endpoint	Purpose
`POST /incidents` `{"scenario_id": "..."}`	runs triage→diagnosis→planning, returns state at `awaiting_approval`
`GET /incidents/{id}`	full incident state
`GET /approvals/{id}`	lightweight poll target (used by n8n's Wait loop)
`POST /incidents/{id}/decision` `{"decision": "approve|reject|modify", "reviewer": "...", "modified_goal": "..."}`	the HITL checkpoint (JSON; used by n8n/curl)
`POST /incidents/{id}/decision-form` (HTML form fields)	the HITL checkpoint (used by the dashboard's Approve/Modify/Reject buttons; redirects back to the report page)
`POST /notify`	mock notification sink (stands in for Slack/PagerDuty)
`GET /dashboard`	human-readable live list of every incident, including past ones reloaded from `runs/` on startup
`GET /incidents/{id}/report`	human-readable live view of one incident, including the HITL form when applicable

n8n workflow (alerting & routing)

n8n is free/open-source and fully self-hosted here — no n8n.io account needed. The owner email+password n8n asks for on first run is purely local to your own container (n8n's own user-management DB), not an external service.

Start a dedicated n8n instance (won't collide with any n8n you already have running elsewhere — it uses port 5679, not the default 5678):

```
docker compose up -d          # starts n8n at http://localhost:5679
```

First time only: open http://localhost:5679, it'll prompt you to create that local owner account, then land you on an empty workflow list.

Import the workflow: the "..." menu (top right of the editor) → *Import from file...* → select `n8n/incident_response_workflow.json`. All 11 nodes and their connections should appear immediately. Click **Publish** (top right) so its production webhook is always listening — without publishing, n8n only listens for one call at a time in "test" mode.

Verified working config: the workflow's `Config` node already points `coordinator_base` at `http://host.docker.internal:8110` — required because n8n runs inside Docker, where `localhost` refers to the *container*, not your host machine where the coordinator API actually listens. (This was confirmed by actually running it: pointing it at `localhost:8110` fails with "the service refused the connection" on the `Start Incident` node — a good one to know about if you ever repoint it.)

Trigger it:

```
curl -X POST http://localhost:5679/webhook/incident-alert \
  -H "Content-Type: application/json" -d '{"scenario_id": "checkout-latency-spike"}'
```

This returns immediately once routing/notification is done (e.g. `{"incident_id":"INC-…","status":"awaiting_approval","routed_to":"pager-oncall"}`) while the workflow keeps running in the background, polling `/approvals/{id}` every 5s until a human decides — approve/reject via `POST /incidents/{id}/decision` as above, or the live dashboard.

To see the incident data flowing through n8n itself (not just the coordinator's own dashboard): open the **Executions** tab in n8n, click the run, then click any node in the execution graph — the side panel shows exactly what that node received as input and produced as output (e.g. the `Alert Webhook` node shows the raw HTTP headers/body of the alert that came in; `Compute Routing` shows the severity-based channel decision it computed). Every node in a finished execution is inspectable this way.

To route the CLI demo's alert through n8n instead of posting directly to the coordinator, set `N8N_ALERT_WEBHOOK_URL=http://localhost:5679/webhook/incident-alert` in `.env`.

LangSmith tracing

With `LANGSMITH_API_KEY` set in `.env`, every incident produces one nested run tree in your `LANGSMITH_PROJECT` (default `incident-response-adk`) — expand it to see the triage/diagnosis/remediation/execution/post-mortem stages in order, tagged `incident:<id>`, including which calls ran in degraded (fallback) mode.

Tests

```
pip install -r requirements-dev.txt
pytest tests/ -v
```

Covers the simulated data layer (KB ranking, chaos injection), schema validation, the deterministic fallback reasoning for all 4 agents, and the coordinator's resilience/HITL-goal-selection logic — all runnable without any API key or live agent servers.

Project layout

```
incident_response/
  agents/            monitoring / diagnostic / remediation / postmortem LlmAgents
                      (each with an in-agent deterministic fallback)
  a2a_server.py       launches one agent as a standalone A2A (Starlette) server
  coordinator.py       orchestration, HITL gate, retry/fallback, execution
  api/server.py         FastAPI surface used by n8n and the CLI demo
  api/report.py          renders the live HTML dashboard/incident-report pages
  data/                 simulated telemetry, knowledge base, remediation catalog,
                         canned incident scenarios (with chaos injection)
  schemas.py             Pydantic contracts agents hand off between each other
  tracing.py              LangSmith helpers
  run_incident.py          CLI demo entrypoint
n8n/incident_response_workflow.json   importable alert intake/routing workflow
docker-compose.yml                    dedicated n8n instance (port 5679)
tests/                                pytest suite (no API key required)
ARCHITECTURE.md                        full design writeup + diagrams
```

Known limitations

See `ARCHITECTURE.md` §9 for the full discussion (fallback ranking quality, per-process chaos budget, in-memory incident store, no auth on approvals).